

Práctica final: Eliminar una propiedad

1. Contexto

El catálogo **Realtor** ya permite completar las siguientes operaciones sobre las propiedades:

- Listar propiedades.
- Consultar el detalle de una propiedad.
- Registrar una nueva propiedad.
- Actualizar una propiedad existente.

Estas funcionalidades fueron desarrolladas mediante las especificaciones 003 a 016.

Para completar el ciclo CRUD, falta implementar la eliminación de una propiedad. En esta práctica final, el alumno deberá diseñar e implementar esta funcionalidad de extremo a extremo, incluyendo el backend y el frontend Blazor, siguiendo el flujo obligatorio de **Spec-Driven Development** definido en el repositorio.

2. Objetivo funcional

Como operador del catálogo Realtor, quiero eliminar una propiedad existente mediante su identificador, para retirar del sistema aquellos inmuebles que ya no deben publicarse ni estar disponibles para consulta.

3. Alcance

La solución deberá cubrir las siguientes capas de la aplicación.

3.1. Backend — Minimal API

Implementar un endpoint público que permita eliminar una propiedad mediante su identificador:

DELETE /api/properties/{id}

El endpoint deberá seguir la arquitectura de **vertical slices** y las convenciones existentes en el proyecto.

3.2. Frontend — Blazor Web App

Incorporar una acción de eliminación en la interfaz de usuario, disponible desde el listado o desde la página de detalle de la propiedad.

La interfaz deberá:

- Invocar el endpoint mediante el cliente Refit tipado.
 - Solicitar confirmación explícita antes de ejecutar la operación.
 - Informar al usuario si la operación fue exitosa o si ocurrió un error.
 - Actualizar la vista después de completar la eliminación.
-

4. Requisitos funcionales

RF-01 — Eliminación por identificador

El sistema **DEBE** eliminar la propiedad cuyo identificador coincida con el valor recibido en la ruta.

RF-02 — Respuesta exitosa

Cuando la propiedad se elimine correctamente, el sistema **DEBE** responder con:

204 No Content

La respuesta no deberá incluir contenido en el cuerpo.

RF-03 — Propiedad inexistente

Si no existe una propiedad con el identificador solicitado, el sistema **DEBE** responder con:

404 Not Found

La operación no deberá producir cambios ni efectos secundarios.

RF-04 — Identificador inválido

Si el identificador recibido no tiene un formato válido, el sistema **DEBE** responder con:

400 Bad Request

RF-05 — Persistencia de la eliminación

Después de una eliminación exitosa, la propiedad **NO DEBE** aparecer en consultas posteriores, incluyendo:

- El listado de propiedades.
- La consulta de detalle por identificador.

RF-06 — Integridad del catálogo

La eliminación de una propiedad **NO DEBE** modificar ni eliminar otras propiedades del catálogo.

RF-07 — Acceso público

El endpoint deberá ser público y no requerirá autenticación, manteniendo la coherencia con los demás endpoints existentes.

RF-08 — Confirmación en la interfaz

Antes de ejecutar la eliminación, el frontend **DEBE** solicitar una confirmación explícita al usuario.

RF-09 — Actualización de la interfaz

Después de una eliminación exitosa, el frontend **DEBE**:

- Retirar la propiedad de la vista actual.
- Evitar que el usuario continúe interactuando con la propiedad eliminada.
- Mostrar un mensaje de confirmación.

Si la operación falla, deberá mostrarse un mensaje de error comprensible y la interfaz deberá permanecer en un estado consistente.

5. Escenarios de aceptación

Escenario 1 — Eliminar una propiedad existente

Dado que existe una propiedad con un identificador válido,
cuando se envía una solicitud para eliminarla,
entonces el sistema responde **204 No Content**
y la propiedad deja de estar disponible en el catálogo.

Escenario 2 — Intentar eliminar una propiedad inexistente

Dado que no existe una propiedad con el identificador solicitado,
cuando se intenta eliminarla,
entonces el sistema responde `404 Not Found`
y no se modifica ninguna propiedad.

Escenario 3 — Enviar un identificador inválido

Dado que el identificador recibido tiene un formato inválido,
cuando se intenta realizar la eliminación,
entonces el sistema responde `400 Bad Request`.

Escenario 4 — Mantener intactas las demás propiedades

Dado que el catálogo contiene varias propiedades,
cuando se elimina una de ellas,
entonces las demás propiedades permanecen disponibles y sin modificaciones.

Escenario 5 — Eliminar desde el listado

Dado que el usuario se encuentra en el listado de propiedades,
cuando selecciona la acción **Eliminar** y confirma la operación,
entonces la propiedad se elimina, desaparece de la vista
y se muestra un mensaje de éxito.

Escenario 6 — Cancelar la eliminación

Dado que se muestra el diálogo de confirmación,
cuando el usuario cancela la operación,
entonces no se envía ninguna solicitud de eliminación
y la propiedad permanece disponible.

Escenario 7 — Error durante la eliminación

Dado que el usuario confirma la eliminación,
cuando el backend devuelve un error,
entonces la propiedad permanece visible
y la interfaz muestra un mensaje indicando que la operación no pudo completarse.

6. Casos límite

La especificación deberá contemplar, como mínimo, los siguientes casos:

- Intentar eliminar una propiedad que ya fue eliminada por otra solicitud. La segunda operación deberá responder **404 Not Found**.
 - Ejecutar dos eliminaciones concurrentes sobre el mismo identificador.
 - Eliminar una propiedad que tenga un valor asociado en **imageUrl**.
 - Recibir una respuesta de error o perder la conexión durante la operación.
 - Evitar que el usuario envíe varias solicitudes de eliminación mientras la primera continúa en proceso.
 - Intentar consultar el detalle de una propiedad después de haberla eliminado.
-

7. Decisiones que el alumno deberá resolver

Durante la creación de la especificación y del plan técnico, el alumno deberá analizar, justificar y documentar las siguientes decisiones.

7.1. Tipo de eliminación

Definir si la solución utilizará:

- Eliminación física.
- Eliminación lógica mediante *soft delete*.

La decisión deberá justificarse en función de los requisitos funcionales y reflejarse en los documentos técnicos correspondientes, especialmente en **data-model.md** cuando este artefacto sea necesario.

Independientemente de la estrategia elegida, una propiedad eliminada no deberá aparecer en las consultas normales del catálogo.

7.2. Manejo de la imagen asociada

Definir qué ocurrirá con el recurso indicado mediante **imageUrl**.

La especificación deberá establecer explícitamente si:

- Solo se elimina el registro de la propiedad.
- También se elimina el archivo o recurso externo.
- La eliminación del recurso queda fuera del alcance de esta iniciativa.

7.3. Estrategia de confirmación

Definir el mecanismo de confirmación utilizado en la interfaz, por ejemplo:

- Diálogo modal.
- Componente de confirmación reutilizable.
- Confirmación integrada en la tarjeta o página de detalle.

La solución deberá prevenir eliminaciones accidentales y mantener una experiencia de usuario clara.

7.4. Ubicación de la acción

Definir si la acción de eliminación estará disponible desde:

- El listado de propiedades.
- La página de detalle.
- Ambas ubicaciones.

La decisión deberá justificarse en la especificación.

8. Entregables obligatorios

Se deberá crear la iniciativa:

017-blazor-delete-property

La implementación deberá respetar el flujo completo de Spec-Driven Development:

```
speckit.specify
  ↓
speckit.plan
  ↓
speckit.tasks
  ↓
speckit.implement
```

Como mínimo, deberán generarse los siguientes artefactos canónicos:

```
specs/017-blazor-delete-property/spec.md
specs/017-blazor-delete-property/plan.md
specs/017-blazor-delete-property/tasks.md
```